PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: Teste de Software

Professor: Cleiton Tavares

Trabalho prático

Detecção de Bad Smells e Refatoração Segura

Aluno: Marcos Alberto Ferreira Pinto

Matrícula: 857547

Repositório: https://github.com/marcosffp/bad-smells-js-refactoring

Outubro de 2026

1. Análise de Smells

Antes de refatorar, li o arquivo `src/ReportGenerator.js`. Ele tem uma classe com um único método, `generateReport`, e identifiquei três tipos de smell nele. O ESLint apontou a complexidade dessa mesma função (primeiro smell) e um `if` que podia ser fundido com outro (segundo); o terceiro só apareceu na leitura manual. As linhas citadas são do arquivo original.

1.1 Long Method (linhas 11 a 68)

O `generateReport` (o chamado God Method) tem cerca de 58 linhas e reúne várias tarefas: monta o cabeçalho, percorre os itens, monta o rodapé, trata dois formatos (CSV e HTML) e dois perfis (ADMIN e USER) e ainda soma o total. É um mau cheiro porque quem precisa mudar só o rodapé HTML tem que ler o método inteiro. Os próprios comentários de seção ("Seção do Cabeçalho", "do Corpo", "do Rodapé") mostram blocos que deveriam ser métodos. Para os testes, o risco é que só dá para validar o método por inteiro: não dá para testar o cabeçalho, uma linha ou o rodapé isoladamente.

1.2 Switch Statements (cadeia if/else por formato repetida)

O smell aqui é a repetição: a mesma estrutura de decisão por formato, `if CSV / else if HTML`, aparece quatro vezes: no cabeçalho, no corpo (duas vezes, uma para cada perfil) e no rodapé (linhas 16 a 65). É um mau cheiro porque para criar um formato novo, como JSON, eu teria que editar todos esses pontos do mesmo método, e um tipo ou perfil desconhecido é ignorado, sem aviso. O risco nos testes é que o número de caminhos cresce com a combinação formato, perfil e faixa de valor, e cobrir todos exige muitos casos. Dentro do laço ainda há quatro níveis de decisão, o que eleva a complexidade cognitiva da função.

1.3 Duplicated Code (linhas 36/47, 40/50 e 37/41/48/51)

Duplicação é a mesma estrutura em mais de um lugar, não só texto idêntico. A linha CSV está escrita duas vezes, idêntica. A linha HTML também, com a única diferença do estilo, e o `total += item.value` se repete quatro vezes. É um mau cheiro porque mudar o formato de uma linha exige editar dois lugares, e se eu esquecer um, o relatório do administrador passa a ser diferente do relatório do usuário comum. Nos testes, cada cópia precisa ser coberta separadamente, e um defeito em uma delas passa despercebido se só a outra for exercitada. Como não há hierarquia de classes, o Pull Up Method não se aplica, e o caminho é o Extract Method. Também anotei os números mágicos `1000` e `500` (linhas 30 e 45), que tratei na refatoração.

2. Relatório da Ferramenta

O projeto já vinha com o ESLint 8.57 e o `eslint-plugin-sonarjs` 0.21 nas dependências. Troquei o `.eslintrc.json` pelo do enunciado, com as regras recomendadas do SonarJS e o limite de complexidade cognitiva em 15. Na primeira execução (`npx eslint src/`), a ferramenta apontou 2 erros, ambos no arquivo original.

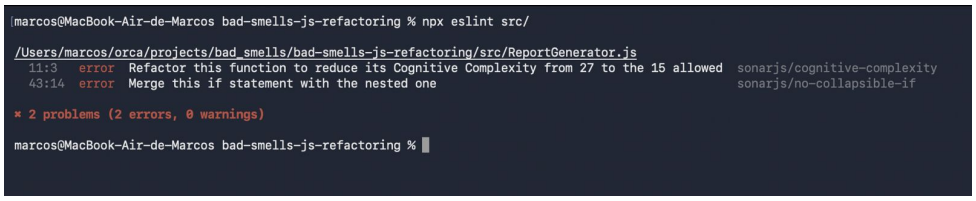


Figura 1: primeira execução do ESLint, com 2 erros em `ReportGenerator.js`.

Linha	Regra do ESLint	Smell correspondente
11	`sonarjs/cognitive-complexity` (erro): complexidade 27, limite 15	Long Method e Switch Statements (decisões repetidas e aninhadas)
43	`sonarjs/no-collapsible-if` (erro)	Reforça o Switch Statements: o if interno do ramo USER podia ser fundido com o externo

A comparação com a análise manual mostra o que a ferramenta acrescentou e o que ela não vê. À mão eu só tinha estimado que a complexidade era "alta"; o SonarJS deu o número (27) e uma meta (15) para a refatoração, e ainda achou o `no-collapsible-if` da linha 43, que eu não tinha percebido. Em contrapartida, ele não apontou a duplicação, porque o `no-identical-functions` só compara funções inteiras idênticas e aqui a duplicação é de trechos dentro de um método, nem os números mágicos. Por isso a leitura manual continuou necessária.

3. Processo de Refatoração

Antes de refatorar, a suíte precisava estar verde, porque o enunciado a usa como rede de segurança. No repositório como veio, o `npm test` não rodava (o Jest não carregava o `import` dos módulos ES) e, depois de ligar essa opção do Node no script do `package.json`, 1 dos 5 testes falhava por um espaço sobrando em `<tr >`, que o código gerava e o teste não esperava. Para conseguir dar continuidade ao trabalho, precisei fazer essas duas alterações no código original: ajustei o script e corrigi o espaço no código, sem tocar no teste. Só então a suíte ficou 100% verde (5 de 5), e essa foi a linha de base.

O smell mais crítico foi o Switch Statements do laço do `generateReport`: é o que mais pesa na complexidade (27), junta a duplicação e é o que mais dificulta estender o código. Fiz a refatoração numa cópia, `src/ReportGenerator.refactored.js`, e criei `tests/ReportGenerator.refactored.test.js` mudando só o import da primeira linha. O teste ficou em `tests/`, e não em `__tests__/` como cita o enunciado, porque é a pasta de testes que este repositório já usa. Refatorei em passos pequenos, rodando `npm test` e fazendo um commit a cada um.

Antes (ReportGenerator.js, trecho resumido)

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},...`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority
        ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}>...</tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        // mesma linha CSV, total += de novo
      } else if (reportType === 'HTML') {
        // mesma linha HTML, total += de novo
      }
    }
  }
}
```

Depois (ReportGenerator.refactored.js, resumido)

```
generateReport(reportType, user, items) {
  const formatter =
    FORMATTERS.get(reportType) ?? EMPTY_FORMATTER;
  let report = formatter.header(user);
  let total = 0;
  for (const item of items) {
    markPriority(user, item);
    if (!canSee(user, item)) continue;
    const row = formatter.row(item, user,
      isHighlighted(user, item));
    if (row !== '') {
      report += row;
      total += item.value;
    }
  }
  report += formatter.footer(total);
  return report.trim();
}

class CsvFormatter {
  header() { return 'ID,NOME,VALOR,USUARIO\n'; }
  row(item, user) {
    return `${item.id},${item.name},...\n`;
  }
  footer(total) { return `\nTotal,,\n${total},,\n`; }
}

class HtmlFormatter {
  header(user) { return '<html>...' + user.name; }
  row(item, user, highlighted) {
    const style = highlighted
      ? ' style="font-weight:bold;"' : '';
    return `<tr${style}>...</tr>\n`;
  }
  footer(total) { return `</table>...${total}`; }
}
```

Usei quatro técnicas, nesta ordem. Primeiro, **Replace Magic Number with Symbolic Constant**: o 1000 e o 500 viraram `PRIORITY_THRESHOLD` e `USER_MAX_VALUE`, e os perfis e formatos ganharam constantes. Depois, **Extract Method**: tirei cabeçalho, linha e rodapé do método longo, e com isso cada linha e o total passaram a existir uma vez só. Em seguida, **Decompose Conditional**: as regras por perfil viraram `markPriority`, `canSee` e `isHighlighted`. Por fim, **Replace Conditional with**

Polymorphism: troquei a cadeia por formato por duas classes, `CsvFormatter` e `HtmlFormatter`, escolhidas por um `Map`. Um formato novo passa a ser uma classe. Um tipo de relatório desconhecido continua gerando saída vazia (um `EmptyFormatter`), como no original.

Cada técnica ficou em um commit, todos com o `npm test` verde. Como os testes cobrem poucos casos, também comparei o original e o refatorado com 7.200 combinações de entrada (tipos e perfis válidos e inválidos, valores-limite), com o script `scripts/comparar-original-refatorado.mjs` do repositório: a saída foi idêntica em todas. Uma coisa deixei como estava: o `markPriority` ainda altera o objeto recebido (`item.priority = true`), como o original fazia. Mantive isso para não mudar o comportamento observável; esse efeito colateral fica como possível melhoria futura.

4. Validação Final

Rodei o ESLint no arquivo refatorado (`npx eslint src/ReportGenerator.refactored.js`) e ele não aponta nenhum problema (exit code 0), contra os 2 erros do original (Figura 2). A maior complexidade cognitiva entre as funções caiu de 27 para 6. Rodei a suíte completa com `npm test`: os dois arquivos passam, 10 testes em 2 suítes (Figura 3). O teste original não foi modificado e o `ReportGenerator.js` só mudou na correção do espaço descrita na seção 3, antes da refatoração. Tudo foi enviado ao GitHub.

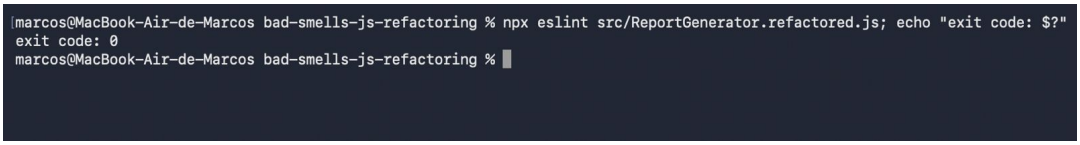


Figura 2: ESLint em `ReportGenerator.refactored.js`, sem erros e sem avisos (exit code: 0).

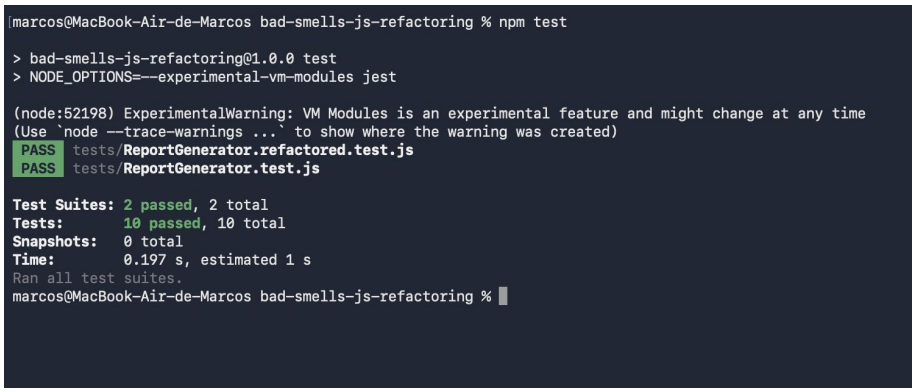


Figura 3: `npm test` com as duas suítes passando.

5. Conclusão

Os testes foram a rede de segurança que permitiu mexer no código com segurança. Como a suíte descrevia o comportamento esperado, pude extrair, mover e reorganizar tudo sabendo na hora se algo quebrasse. O início do trabalho mostrou o valor disso: um dos testes falhava por causa de um espaço no HTML, e sem uma linha de base verde eu não teria como separar esse defeito antigo de um bug que eu mesmo introduzisse. Os testes também têm limite, porque não cobrem tipos ou perfis desconhecidos, e por isso complementei com a comparação em larga escala entre as duas versões.

Reduzir os smells melhorou o software. O ESLint foi de 2 erros para 0, a complexidade saiu de 27 para 6 (o limite era 15), o método longo virou funções pequenas com nomes que dizem o que fazem e a duplicação deixou de existir. Isso torna o código mais fácil de ler, de testar em partes e de estender, já que um formato novo passa a ser uma classe nova em vez de edições espalhadas. A análise estática mostrou onde olhar, mas não viu a duplicação nem os números mágicos, então a leitura manual continuou necessária.